

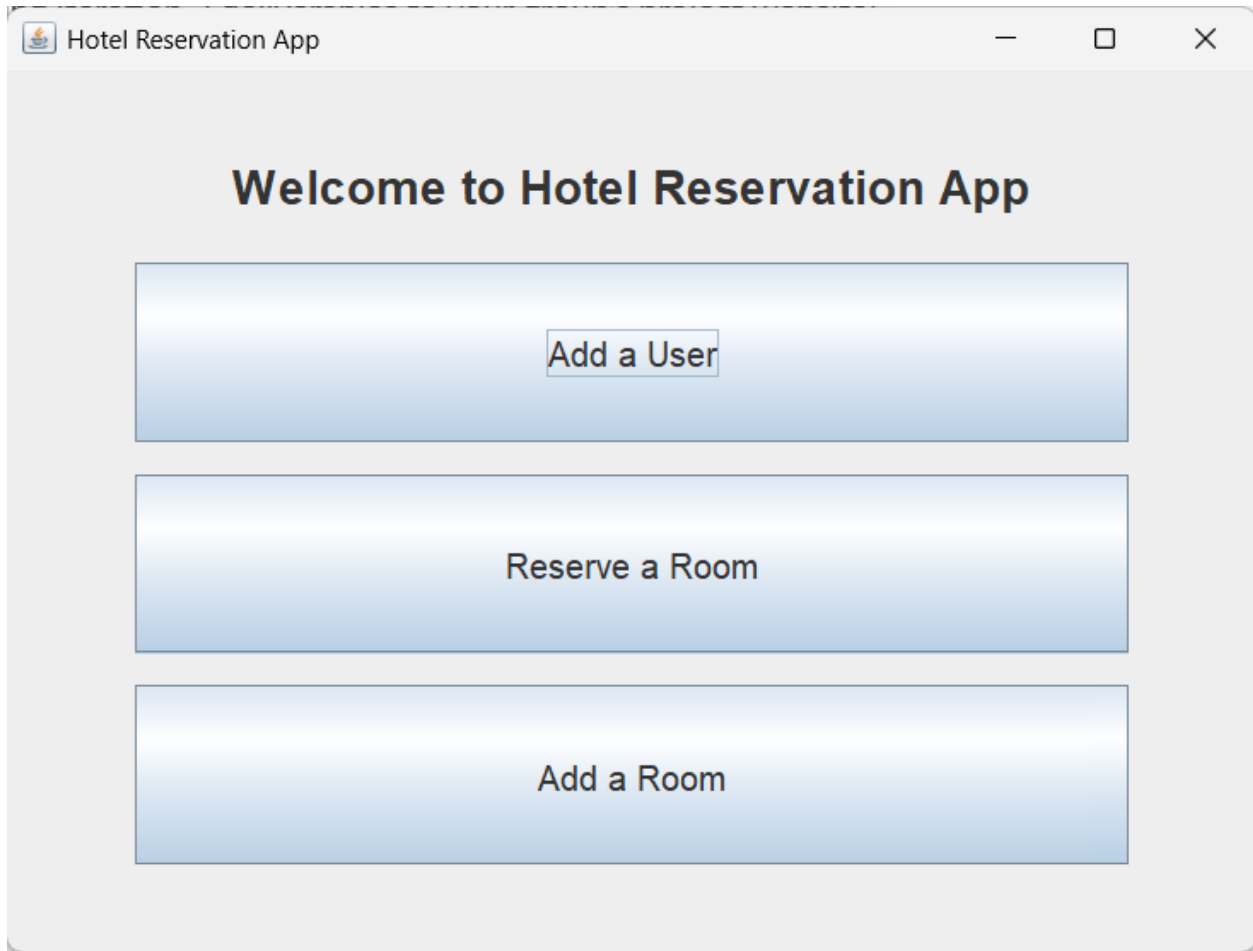
Current Implementation Status

- AddUser, AddRoom, and ReserveRoom backend and UI implemented and connected
- Database to store Users, Rooms, and Reservations that persists when the program is off
- Some other functionality, like searching rooms, has functionality but not yet a UI

Main Roadblocks

- Adding to code that someone else wrote has proved difficult. It takes some time to read the code before you can start working. Maybe we ought to comment more about what our code does.
- Getting used to Git has caused some difficulties, especially when merge conflicts arise.
- Learning how to make functional UI's and Databases has been a challenge.

UI Screenshots



No specific use case. Eventually will be replaced by a login screen and such.

UI: Duane, Matt.

Connected to backend of other use cases.

Hotel Reservation App

Username:

Password:

Full Name:

Phone:

Email:

[← Back to Welcome](#) [Register User](#)

Create Account Use Case – Andrew Hutcheson.

UI: Duane, Matt.

Backend: Duane, Andrew.

Connected with backend.

Hotel Reservation App

Room Number: 0

Guest Name:

Credit Card #:

Check-in (YYYY-MM-DD): YYYY-MM-DD

Check-out (YYYY-MM-DD): YYYY-MM-DD

Calculate Cost

Confirm Reservation

← Back to Welcome

Book a Reservation Use Case – Hannah Ross.

UI: Emmanuel, Duane, Matt.

Backend: Emmanuel, Andrew, Matt.

Connected to backend.

 Hotel Reservation App

Room Number:

Floor Type:

NATURAL

▼

Bed Type:

SINGLE

▼

Smoking:

☐

Daily Rate:

← Back to Welcome

Add Room

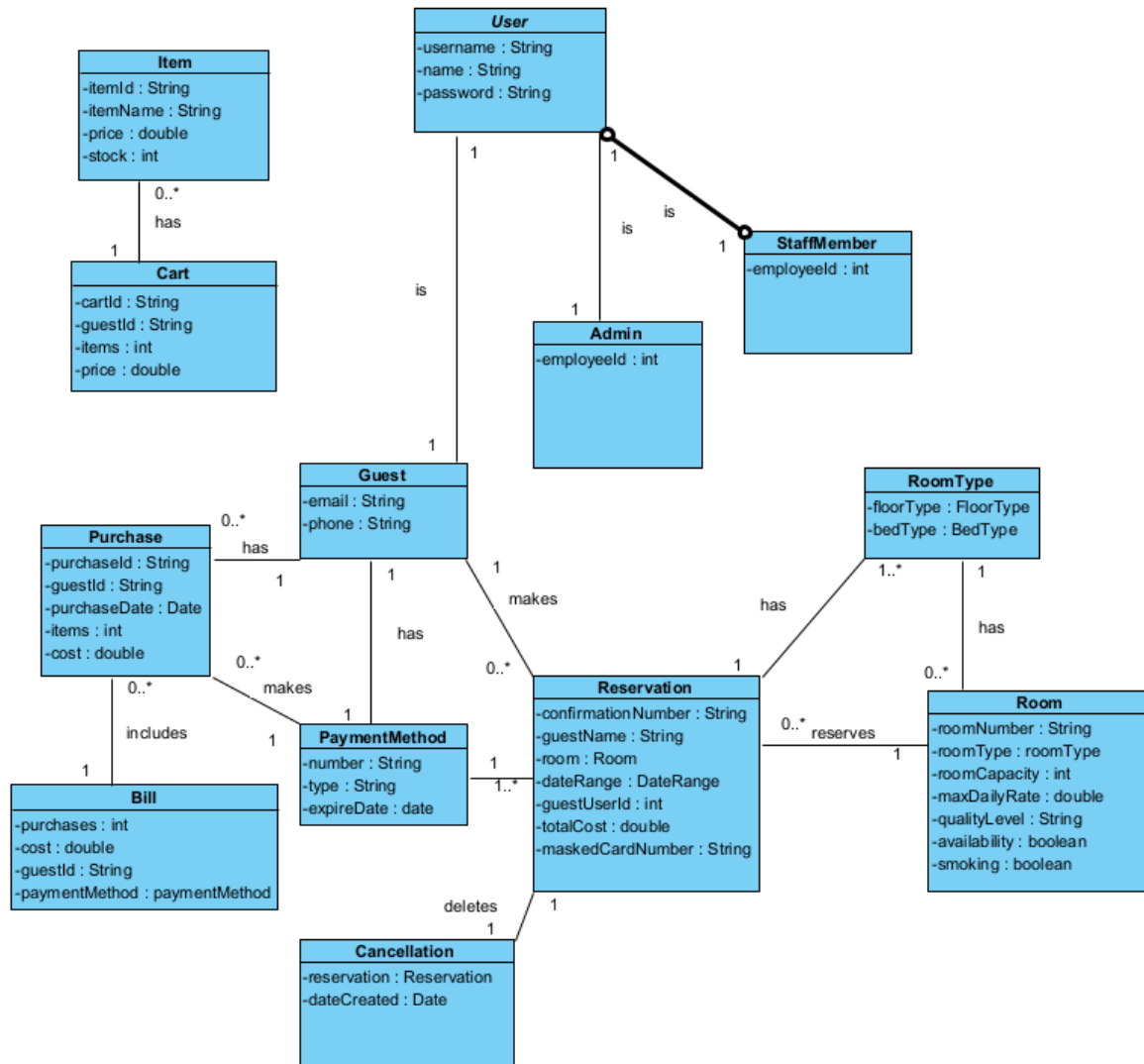
Add Room Use Case – we haven't explicitly written out this use case.

UI: Duane, Hannah, Matt.

Backend: Andrew, Hannah.

Connected to backend.

Domain Model



Deliverable 11

```

classDiagram
    class SearchCriteria {
        -DateRange dateRange
        -RoomType roomType
        -numGuests
        +getDateRange() DateRange
        +getRoomType() RoomType
        +getNumGuests() int
        +searchCriteria() SearchCriteria
    }

    class SearchController {
        -RoomService RS
        -ReservationService ResS
        -UserSession US
        +searchRoom() ListRoom
        +getRoomService() RoomService
        +searchController()
    }

    class RoomService {
        -RoomCatalog catalog
        -SQLiteRoomPersistence roomDb
        -RoomService
        +getCatalog() RoomCatalog
        +searchRoom() ListRoom
        +findRoom() Room
        +getRoom() ListRoom
        +addRoom() boolean
    }

    class ReservationService {
        -List Reservation reservationList
        -ArrayList confirmationSeq
        -SQLiteReservationPersistence sqlite
        -ReservationService
        +calculateOverlap() ListRoom
        +isReserved() boolean
        +addReservation() void
        +string nextConfirmationNumber() String
        +getReservations() ListReservation
        +buildPrevious() ReservationSummary
        +confirmAndSave() String
    }

    class Reservation {
        -confirmationNumber: String
        -localGuest: int
        -guestID: String
        -checkInDate: Date
        -checkOutDate: Date
        -createdDate: Date
        -status: enum Status
    }

    class Room {
        -roomNumber: String
        -floor: int
        -type: String
        -capacity: int
        -pricePerNight: double
        -qualityLevel: String
        -occupancyStatus: enum OS
    }

    class BookingValidation {
        -BookingValidation
        +validateGuestName() String
        +validateAddress() String
        +validateRoomType() String
        +validateLuhn() boolean
    }

    class ReservationSummary {
        -room: Room
        -dateRange: DateRange
        -guestName: String
        -maskerCardNumber: String
        -totalCost: double
        -numberOfNights: int
    }

    class ReservationController {
        -RoomService roomService
        -ReservationService reservationService
        -UserSession userSession
        -ReservationController
        +reserveRoom() ReservationSummary
        +confirmAndSaveReservation() String
        +getRoom() ListRoom
    }

    class RoomType {
        -floorType: FloorType
        -bedType: BedType
        -FloorType: enum
        -BedType: enum
    }

    class Guest {
        -guestID: String
        -status: enum Status
    }

    class UserController {
        +Result: enum
        -UserController userCatalog
        -SQLiteUserPersistence userDb
        -UserController
        +addGuest() Result
        +add Clerk() Result
        +addAdmin() Result
        +findByUsername() User
        +exists() boolean
    }

    SearchCriteria "1..*" -- "1..*" SearchController : reads
    SearchCriteria "1..*" -- "1..*" DateRange : uses
    SearchCriteria "1..*" -- "1..*" RoomType : uses
    SearchController "1..*" -- "1..*" RoomService : uses
    SearchController "1..*" -- "1..*" ReservationService : uses
    SearchController "1..*" -- "1..*" SearchCriteria : uses
    RoomService "1..*" -- "1..*" SearchCriteria : uses
    RoomService "1..*" -- "1..*" ReservationService : uses
    ReservationService "1..*" -- "1..*" Reservation : manages
    ReservationService "1..*" -- "1..*" Room : queries
    ReservationService "1..*" -- "1..*" ReservationSummary : creates
    Reservation "1..*" -- "1..*" Room : reserves
    Reservation "1..*" -- "1..*" BookingValidation : uses
    ReservationSummary "1..*" -- "1..*" ReservationController : uses
    ReservationController "1..*" -- "1..*" SearchCriteria : uses
    ReservationController "1..*" -- "1..*" RoomService : uses
    ReservationController "1..*" -- "1..*" ReservationService : uses
    ReservationController "1..*" -- "1..*" UserSession : uses
    RoomType "1..*" -- "1..*" SearchCriteria : uses
    RoomType "1..*" -- "1..*" Reservation : makes
    Guest "1..*" -- "1..*" Reservation : makes
    Guest "1..*" -- "1..*" UserController : manages
    UserController "1..*" -- "1..*" Reservation : manages
  
```

UML Class Diagram illustrating the structure of a Hotel Management System:

- SearchCriteria**
 - Attributes: `-DateRange dateRange`, `-RoomType roomType`, `-numGuests`
 - Operations: `+getDateRange() : DateRange`, `+getRoomType() : RoomType`, `+getNumGuests() : int`, `+searchCriteria() : SearchCriteria`
- SearchController**
 - Attributes: `-RoomService RS`, `-ReservationService ResS`, `-UserSession US`
 - Operations: `+searchRoom() : ListRoom`, `+getRoomService() : RoomService`, `+searchController()`
- RoomService**
 - Attributes: `-RoomCatalog catalog`, `-SQLiteRoomPersistence roomDb`, `-RoomService`
 - Operations: `+getCatalog() : RoomCatalog`, `+searchRoom() : ListRoom`, `+findRoom() : Room`, `+getRoom() : ListRoom`, `+addRoom() : boolean`
- ReservationService**
 - Attributes: `-List Reservation reservationList`, `-ArrayList confirmationSeq`, `-SQLiteReservationPersistence sqlite`, `-ReservationService`
 - Operations: `+calculateOverlap() : ListRoom`, `+isReserved() : boolean`, `+addReservation() : void`, `+string nextConfirmationNumber() : String`, `+getReservations() : ListReservation`, `+buildPrevious() : ReservationSummary`, `+confirmAndSave() : String`
- Reservation**
 - Attributes: `-confirmationNumber : String`, `-localGuest : int`, `-guestID : String`, `-checkInDate : Date`, `-checkOutDate : Date`, `-createdDate : Date`, `-status : enum Status`
- Room**
 - Attributes: `-roomNumber : String`, `-floor : int`, `-type : String`, `-capacity : int`, `-pricePerNight : double`, `-qualityLevel : String`, `-occupancyStatus : enum OS`
- BookingValidation**
 - Attributes: `-BookingValidation`
 - Operations: `+validateGuestName() : String`, `+validateAddress() : String`, `+validateRoomType() : String`, `+validateLuhn() : boolean`
- ReservationSummary**
 - Attributes: `-room : Room`, `-dateRange : DateRange`, `-guestName : String`, `-maskerCardNumber : String`, `-totalCost : double`, `-numberOfNights : int`
- ReservationController**
 - Attributes: `-RoomService roomService`, `-ReservationService reservationService`, `-UserSession userSession`, `-ReservationController`
 - Operations: `+reserveRoom() : ReservationSummary`, `+confirmAndSaveReservation() : String`, `+getRoom() : ListRoom`
- RoomType**
 - Attributes: `-floorType : FloorType`, `-bedType : BedType`, `-FloorType : enum`, `-BedType : enum`
- Guest**
 - Attributes: `-guestID : String`, `-status : enum Status`
- UserController**
 - Attributes: `+Result : enum`, `-UserController userCatalog`, `-SQLiteUserPersistence userDb`, `-UserController`
 - Operations: `+addGuest() : Result`, `+add Clerk() : Result`, `+addAdmin() : Result`, `+findByUsername() : User`, `+exists() : boolean`

Relationships:

- SearchCriteria** (1..*) **reads** **SearchController** (1..*)
- SearchCriteria** (1..*) **uses** **DateRange** (1..*)
- SearchCriteria** (1..*) **uses** **RoomType** (1..*)
- SearchController** (1..*) **uses** **RoomService** (1..*)
- SearchController** (1..*) **uses** **ReservationService** (1..*)
- SearchController** (1..*) **uses** **SearchCriteria** (1..*)
- RoomService** (1..*) **uses** **SearchCriteria** (1..*)
- RoomService** (1..*) **uses** **ReservationService** (1..*)
- ReservationService** (1..*) **manages** **Reservation** (1..*)
- ReservationService** (1..*) **queries** **Room** (1..*)
- ReservationService** (1..*) **creates** **ReservationSummary** (1..*)
- Reservation** (1..*) **reserves** **Room** (1..*)
- Reservation** (1..*) **uses** **BookingValidation** (1..*)
- ReservationSummary** (1..*) **uses** **ReservationController** (1..*)
- ReservationController** (1..*) **uses** **SearchCriteria** (1..*)
- ReservationController** (1..*) **uses** **RoomService** (1..*)
- ReservationController** (1..*) **uses** **ReservationService** (1..*)
- ReservationController** (1..*) **uses** **UserSession** (1..*)
- RoomType** (1..*) **uses** **SearchCriteria** (1..*)
- RoomType** (1..*) **makes** **Reservation** (1..*)
- Guest** (1..*) **makes** **Reservation** (1..*)
- Guest** (1..*) **manages** **UserController** (1..*)
- UserController** (1..*) **manages** **Reservation** (1..*)

Powered By Paradigm Community Edition

Matt Freeman's Use Cases

Use Case Name	Check in a Guest
Use Case Author	Matt Freeman
Actor	Hotel Clerk
Preconditions	1. The clerk is logged in.
Postconditions	1. The guest's status is set to "checked-in." 2. A room number is assigned to the guest and that room is set to "occupied."
Main Success Scenario	1. A guest approaches the clerk to check in. 2. The clerk searches for the guest's reservation by name or confirmation number. 3. The system displays the reservation and room specifications. 4. The clerk selects "Check-In." 5. The system displays available rooms matching the reservation's room specifications. 6. The clerk selects "Assign Room" on a chosen room. 7. The system sets the guest to "checked-in" and the room number to "occupied."
Extensions	1. No Reservation Today (step 2) i. The guest does not have a reservation today ii. The system displays an error message 2. No Clean Rooms (step 5) i. No clean rooms matching the specifications can be displayed. ii. The clerk may override this and search for other rooms to assign.

Contract CO1: checkIn

Operation: checkIn(reservationID)

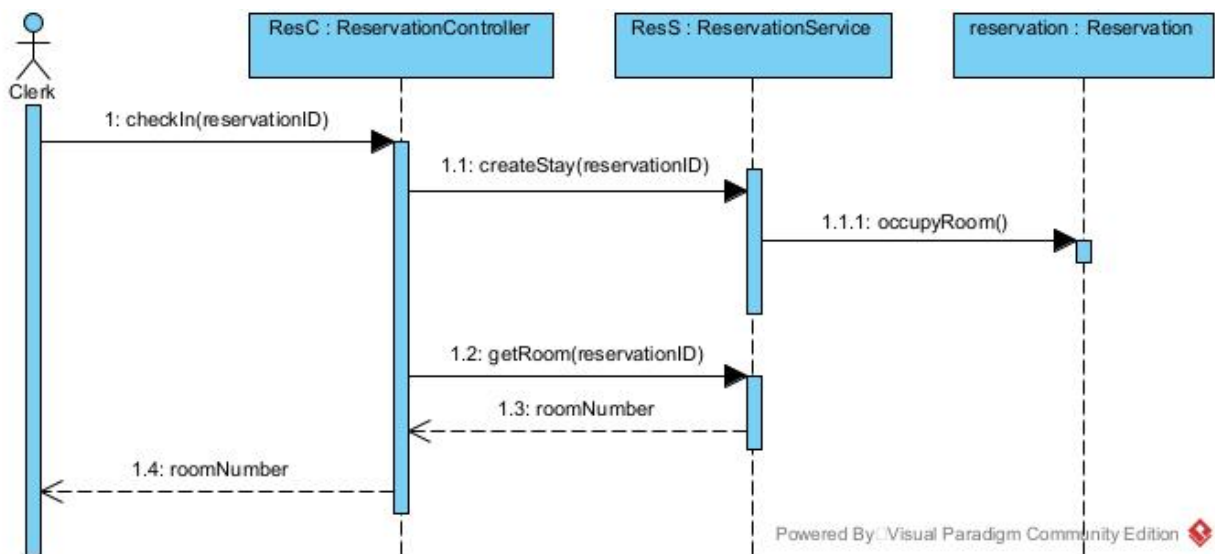
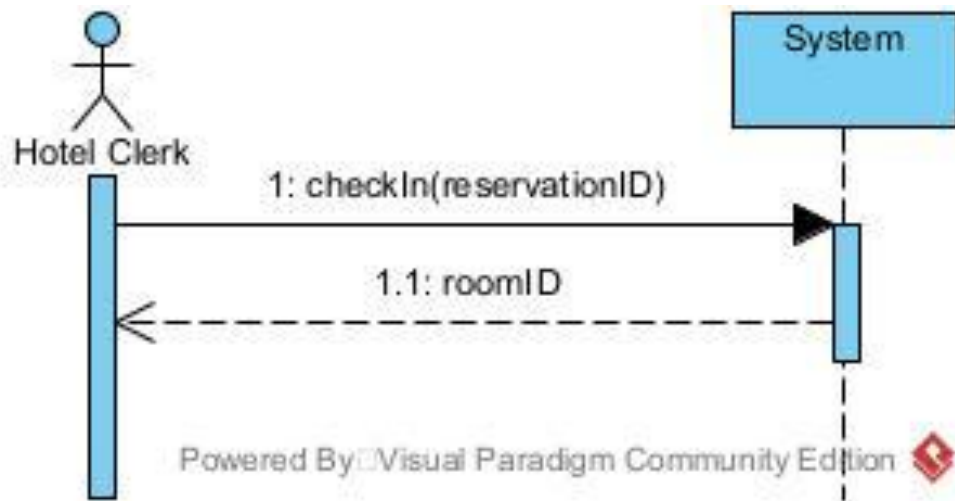
Cross References: Use Case – Check in a Guest

Preconditions

1. The system is online
2. Room and reservation data exist in the system
3. Clerk is logged in
4. Guest has a reservation beginning today

Postconditions

1. Stay object is created, assigned to a room number, and associated with existing Reservation
2. The Reservation's status is updated to "checked-in"
3. The Room is set to "occupied"



Use Case Name	Check out a Guest
Use Case Author	Matt Freeman
Actor	Hotel Clerk
Preconditions	1. The clerk is logged in. 2. The guest's status is set to "checked-in."
Postconditions	1. The guest's status is set to "checked-out." 2. The room number assigned to the guest is set to "cleaning." 3. Combined bill is generated
Main Success Scenario	1. A guest approaches the clerk to check out. 2. The clerk searches for the guest's reservation by name, confirmation number, or room number. 3. The system displays the reservation and room. 4. The clerk selects "Check-Out." 5. The system sets the guest's status to "checked-out" and the room number to "cleaning." 6. The system generates a combined bill for the guest.
Extensions	1. Reservation Doesn't End Today (step 4) i. The system asks for confirmation that the guest wants to check out early. ii. The clerk selects "No," or the clerk selects "Yes" and continues as above.

Contract CO1: checkOut

Operation: checkOut(reservationID)

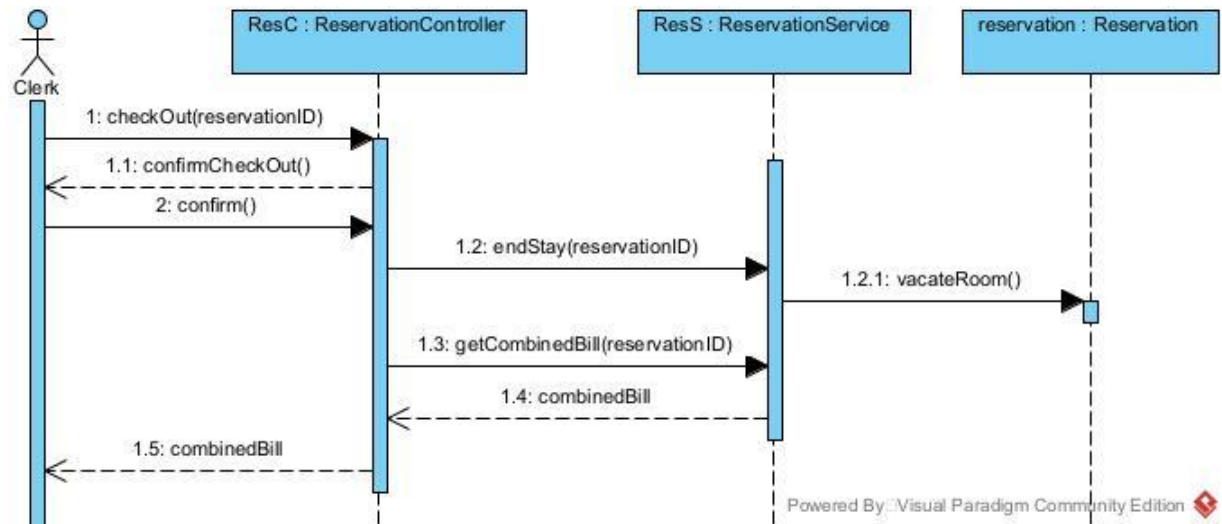
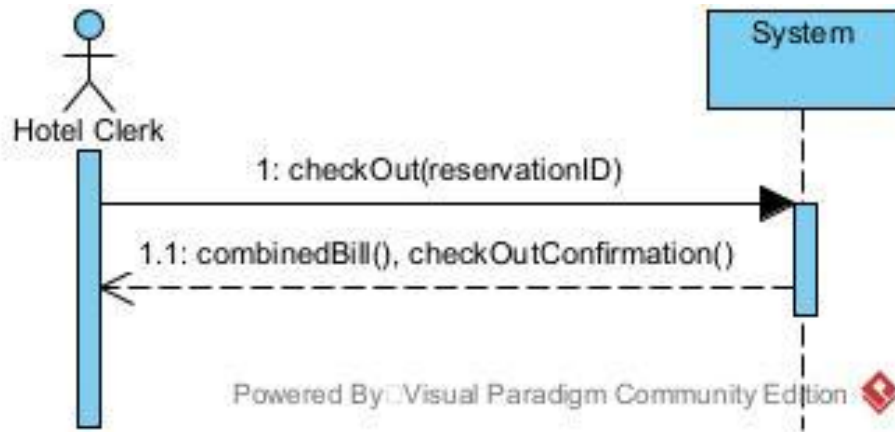
Cross References: Use Case – Check out a Guest

Preconditions

1. The system is online
2. Reservation and Stay objects exist in the system
3. Today is the check-out date, or possible sooner for early check-out
4. Clerk is logged in

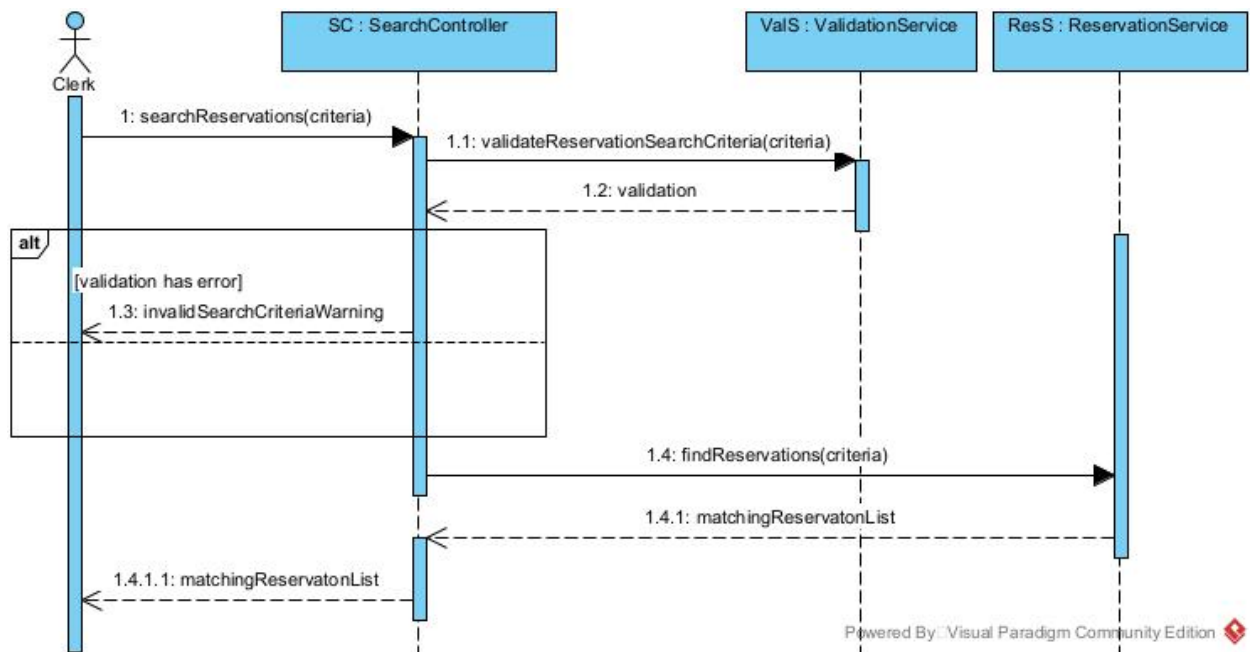
Postconditions

1. Total bill is returned
2. Stay and Reservation are set to "checked-out"
3. The Room is set to "cleaning"



Use Case Name	Search Reservations
Use Case Author	Matt Freeman
Actor	Hotel Clerk
Preconditions	1. The clerk is logged in. 2. Reservation data exists in the system
Postconditions	1. Reservation data is displayed. 2. No reservations are created or modified.
Main Success Scenario	1. The clerk selects “Search Reservations.” 2. The system prompts the clerk to enter search criteria (check-in date, check-out date, confirmation number, etc.). 3. The clerk enters the information. 4. The system validates the input. 5. The system searches for existing reservations matching the criteria. 6. The system displays a list of reservations.
Extensions	1. Invalid Search Criteria (step 4) i. The system detects invalid input (formatting, check-in earlier than check-out, etc.). ii. The system displays an error message and asks the clerk to re-enter the information. 2. No Matching Reservations (step 4) i. The system displays a message indicating there are no existing reservations matching the search criteria.

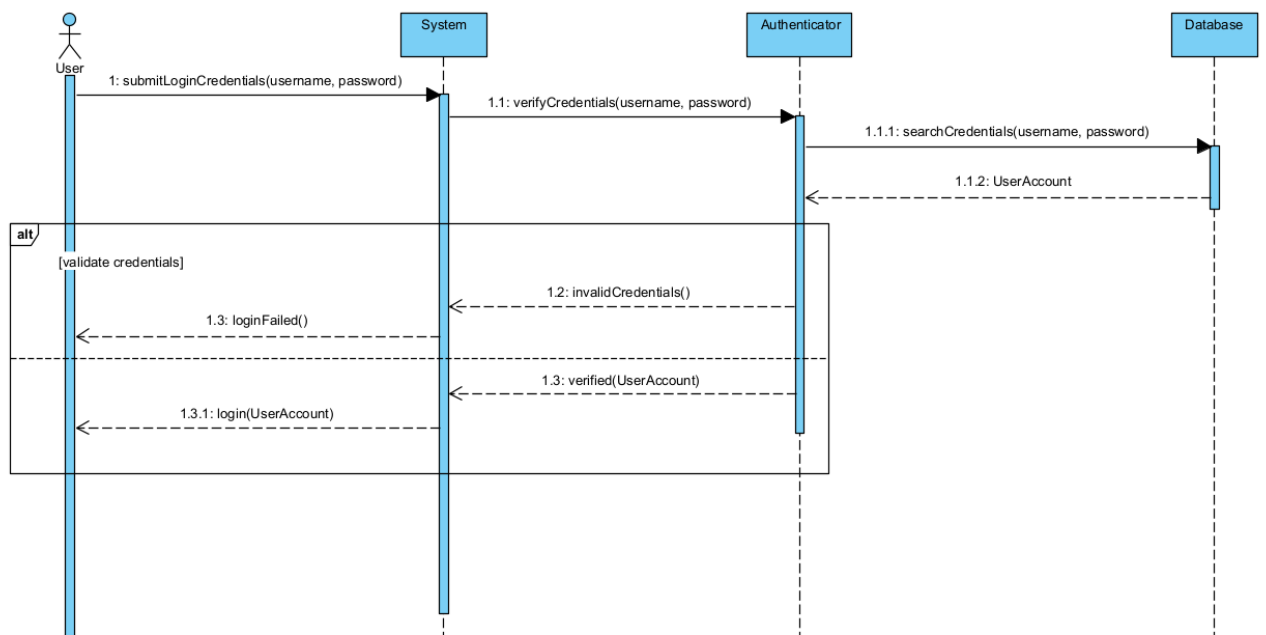
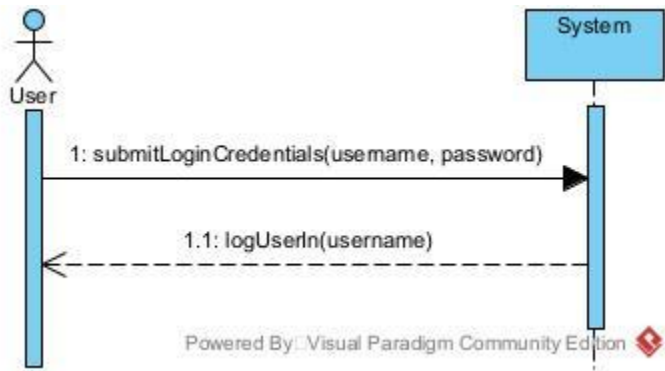
Contract CO1: searchReservations
Operation: searchReservations(checkInDate, checkOutDate, roomTypeID)
Cross References: Use Case – Search Reservations
Preconditions 1. The system is online 2. Reservation data exists in the system 3. Clerk is logged in
Postconditions 1. A list of Reservations matching the search criteria is returned 2. No Reservation objects are created or modified



Andrew Hutcheson's Use Cases

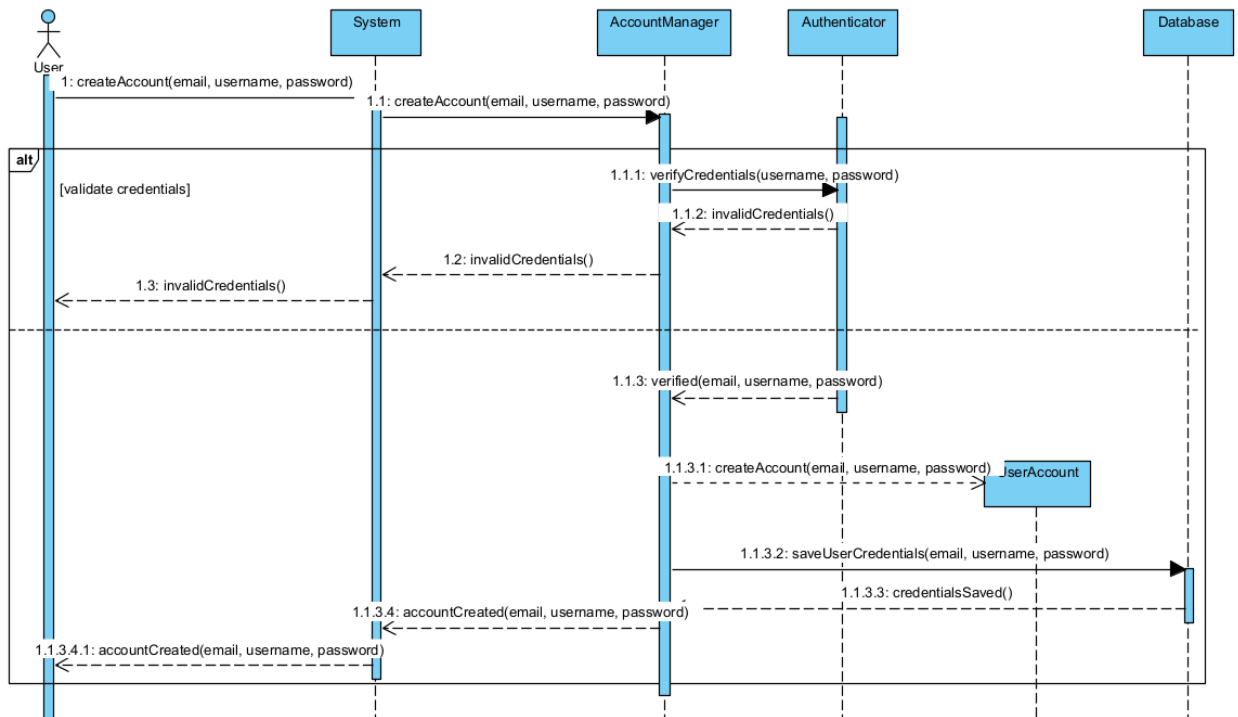
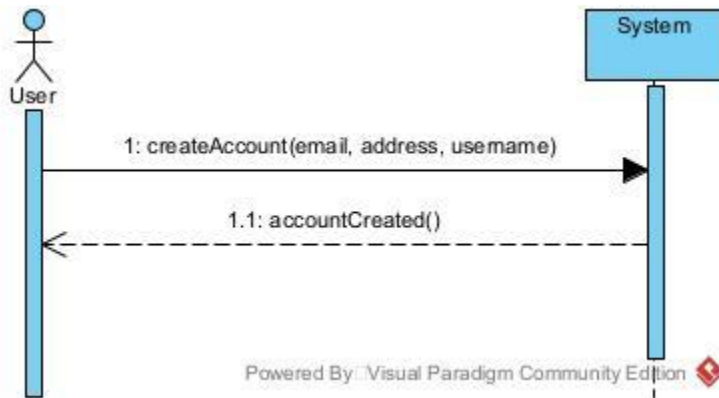
Use Case Name	Login
Use Case Author	Andrew Hutcheson
Actor	Hotel Guest, Staff Member, or Admin
Preconditions	1. The user has a valid account with a username and password. 2. The login system works and is accessible.
Postconditions	1. The user is logged in and has access to their respective dashboard. 2. The user is blocked after a number of failed login attempts.
Main Success Scenario	1. The user navigates to the login page. 2. The user enters their username and password. 3. The system verifies the credentials against saved credentials. 4. The system authenticates the user, verifies their role, and grants access to the authenticated version of the system. 5. The user is redirected to their personal dashboard (different for guests, staff, and admin).
Extensions	1. Invalid Credentials: i. If the user enters incorrect credentials, the system displays an error message. ii. The system prompts them to retry. iii. After a certain amount of tries, the system stops the user from logging. 2. Forgot Password: i. The user can request a password reset and get an email to reset their password. 3. System Down: i. If the system is not working, an error message is displayed.

Contract CO1: logUserIn()
Operation: logUserIn()
Cross References: Use Case - Login
Preconditions 5. The system is online 6. Login page exists and is available
Postconditions 1. The user is on their respective dashboard



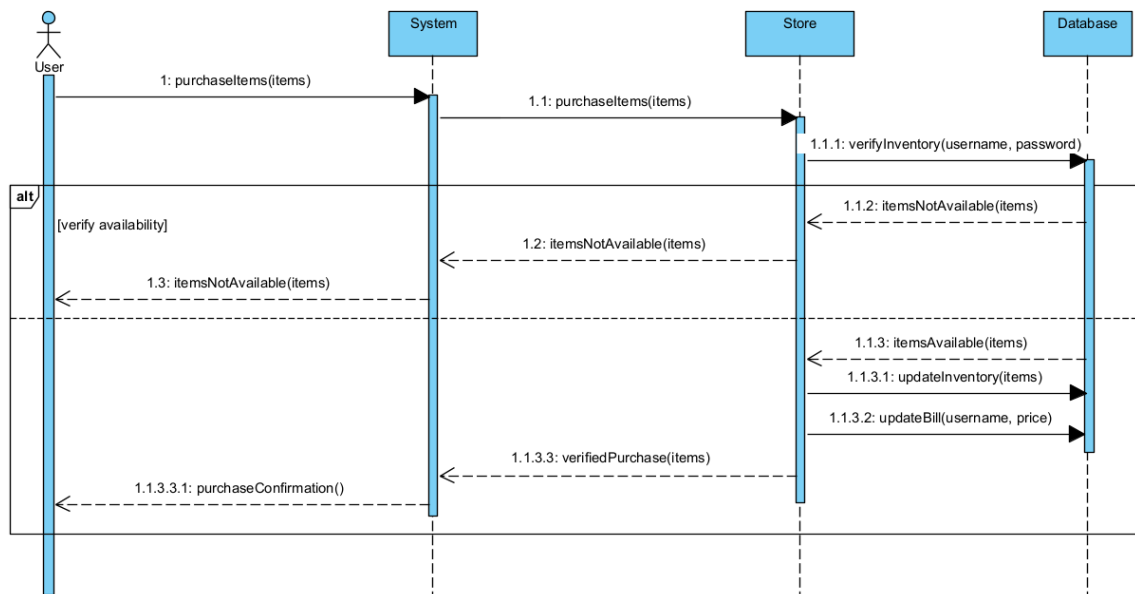
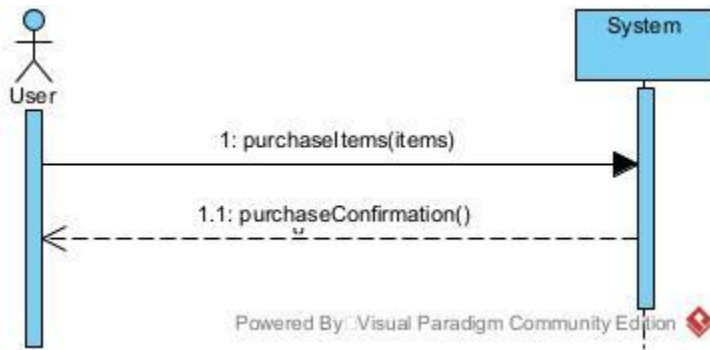
Use Case Name	Create Account
Use Case Author	Andrew Hutcheson
Actor	Hotel Guest
Preconditions	1. Guest does not already have an account associated with their email. 2. The login system works and is accessible.
Postconditions	1. A new guest account is created for the users email and stored in the system. 2. The guest can successfully login with their new credentials.
Main Success Scenario	1. The guest navigates to the create account page. 2. The guest enters their email, password, and any other required credentials. 3. The guest submits the form. 4. The system validates against existing user (checks email doesn't already have an account). 5. The system creates the account and stores the account information. 6. The system redirects the user to the login page.
Extensions	1. Invalid Input: i. If the user's email already has an account or any of their other credentials are invalid. ii. The system prompts the user to either sign in or use another email. 2. System Error: i. If the system is not working, an error message is displayed.

Contract CO2: createAccount()
Operation: createAccount()
Cross References: Use Case – Create Account
Preconditions: 1. User does not have an existing account 2. The system is online
Postconditions 1. User account is successfully created and stored by system User gets confirmation of account being created



Use Case Name	Purchase Items & Generate Combined Bill
Use Case Author	Andrew Hutcheson
Actor	Hotel Guest
Preconditions	1. The guest is logged in. 2. The guest is actively staying at the hotel.
Postconditions	1. The selected items are logged as purchased in the system. 2. The combined bill is updated with new purchases.
Main Success Scenario	1. The guest navigates to the shopping cart. 2. The guest selects some items for purchase. 3. The guest purchases selected items. 4. The system calculates the total number of items purchased. 5. The system adds the total to the guest's hotel bill (created combined bill). 6. The system generates a combined bill (room price + store purchases).
Extensions	1. Guest Not Checked In: i. The system displays an error message telling the guest they must be actively staying at the hotel to purchase items from the store. 2. Item out of stock: i. The system displays an error message telling the guest the items are out of stock.

Contract CO1: purchaseItems()
Operation: purchaseItems()
Cross References: Use Case – purchase Items and Generate Combined Bill
Preconditions 1. The system is online 2. The user is an active guest at the hotel 3. The user has an account already
Postconditions 1. The items are marked as purchased in the system 2. The user's bill is updated to include the purchased items



Hannah Ross's Use Cases

Use Case Name	Book a Reservation
Use Case Author	Hannah Ross
Actor	Hotel Guest
Preconditions	1. Guest is logged in. 2. The system shows available rooms.
Postconditions	1. A reservation is saved. 2. Room is reserved for specified dates.
Main Success Scenario	1. Choose Guest selects desired room. 2. System requests Guest Info (Name, Address, Credit Card, Stay Dates). 3. Guest enters required details. 4. System validates credit card, address, and availability. 5. System calculates stay cost and provides summary of information. 6. Guest approves cost amount and details of reservation. 7. System saves reservation. 8. System displays confirmation number.
Extensions	1. Invalid search criteria (step 4) i. The system detects invalid input (e.g., check-out date earlier than check-in date). ii. The system displays an error message and requests corrected input.

Contract CO1: bookReservation

Operation: bookReservation(reservation)

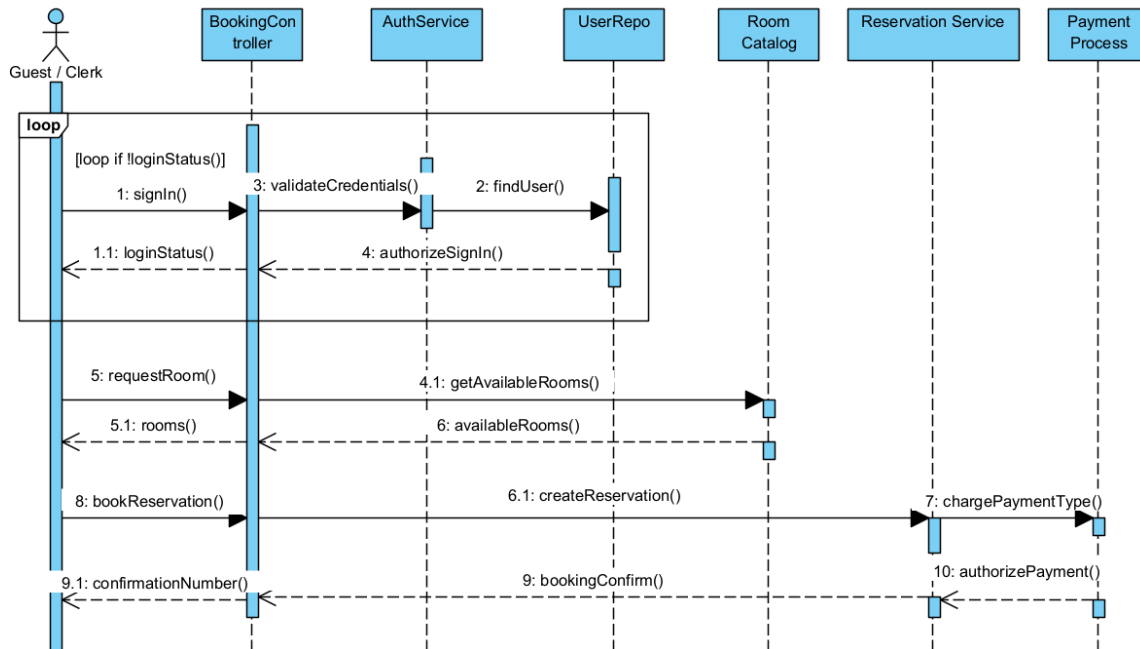
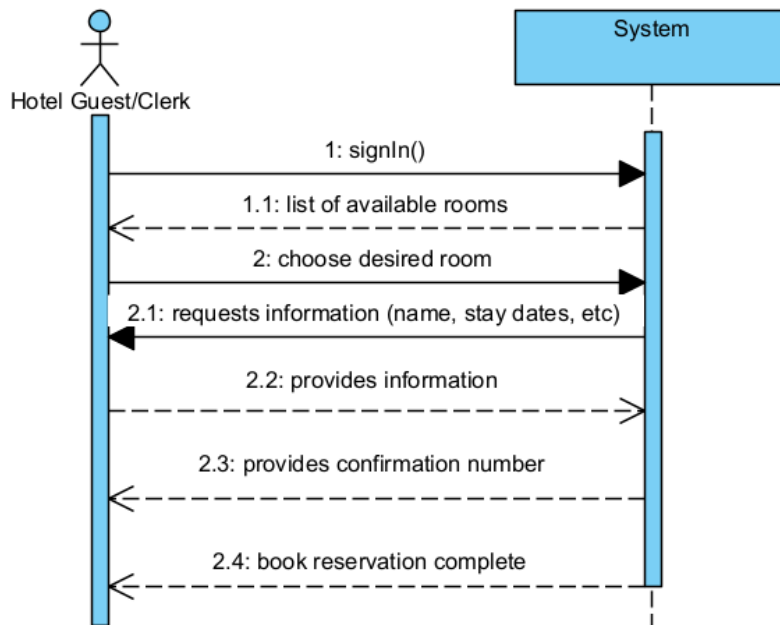
Cross References: Use Case – Sign in, Check in a Guest

Preconditions

1. The system is online
2. Guest is authenticated
3. At least one room = “Available” for requested dates/other details.

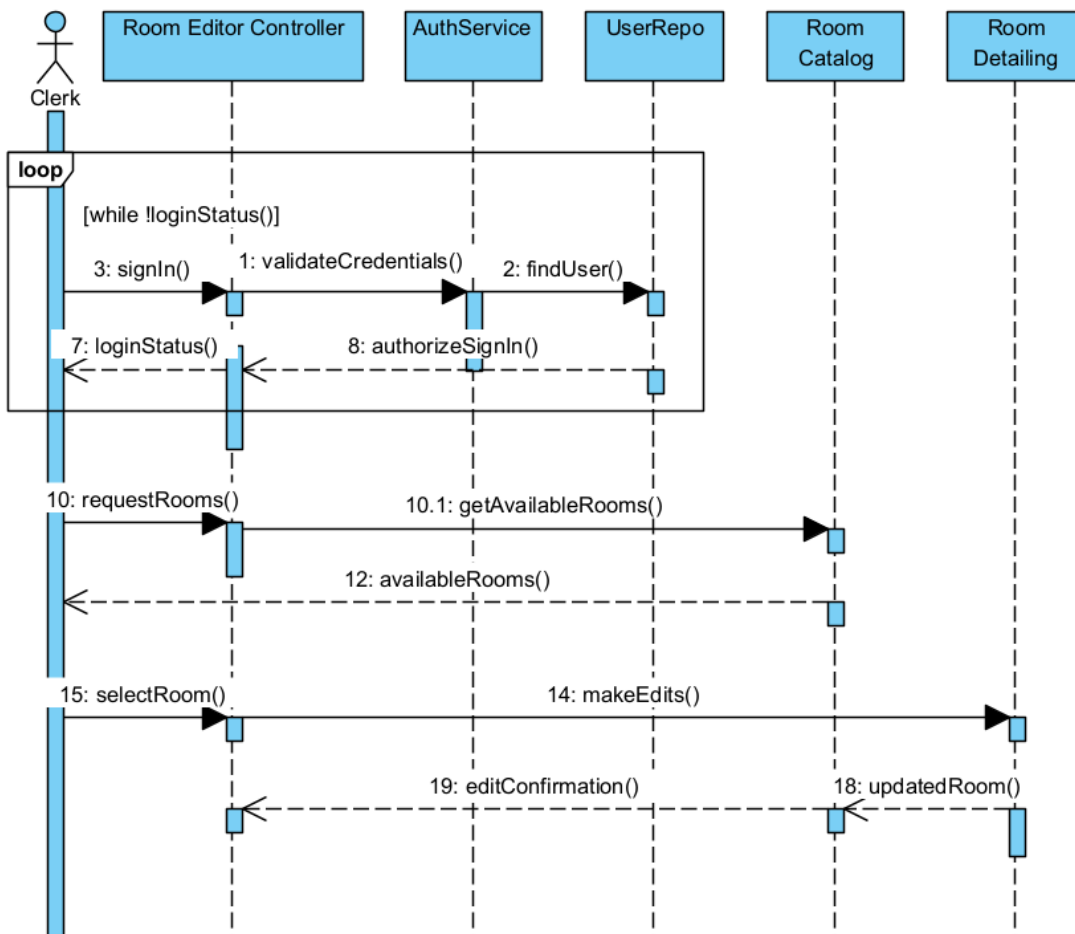
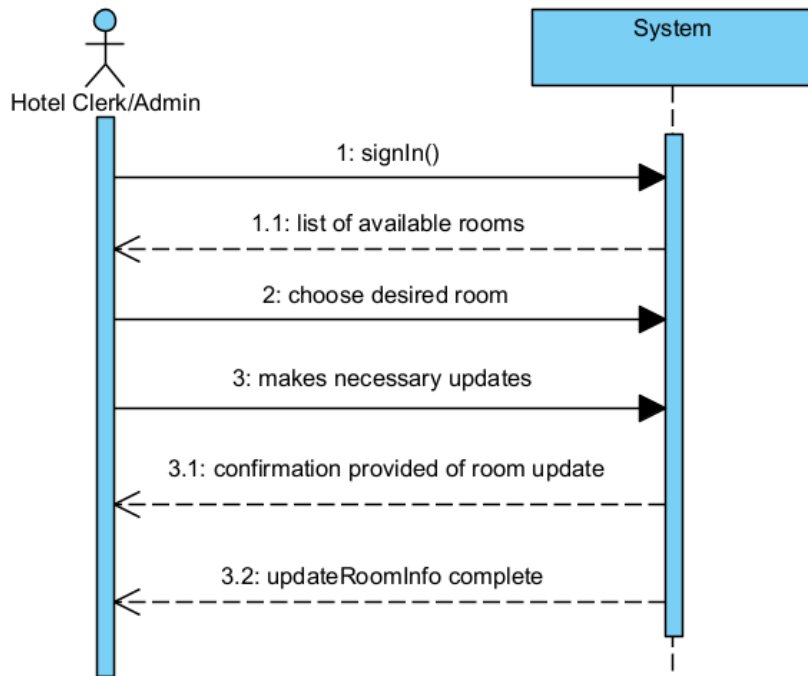
Postconditions

1. The room is selected and moved to “reserved/booked/occupied”
2. The reservation is aligned with the correct guest and room.
3. The confirmation number is generated.
4. The system updates the reservation with details.



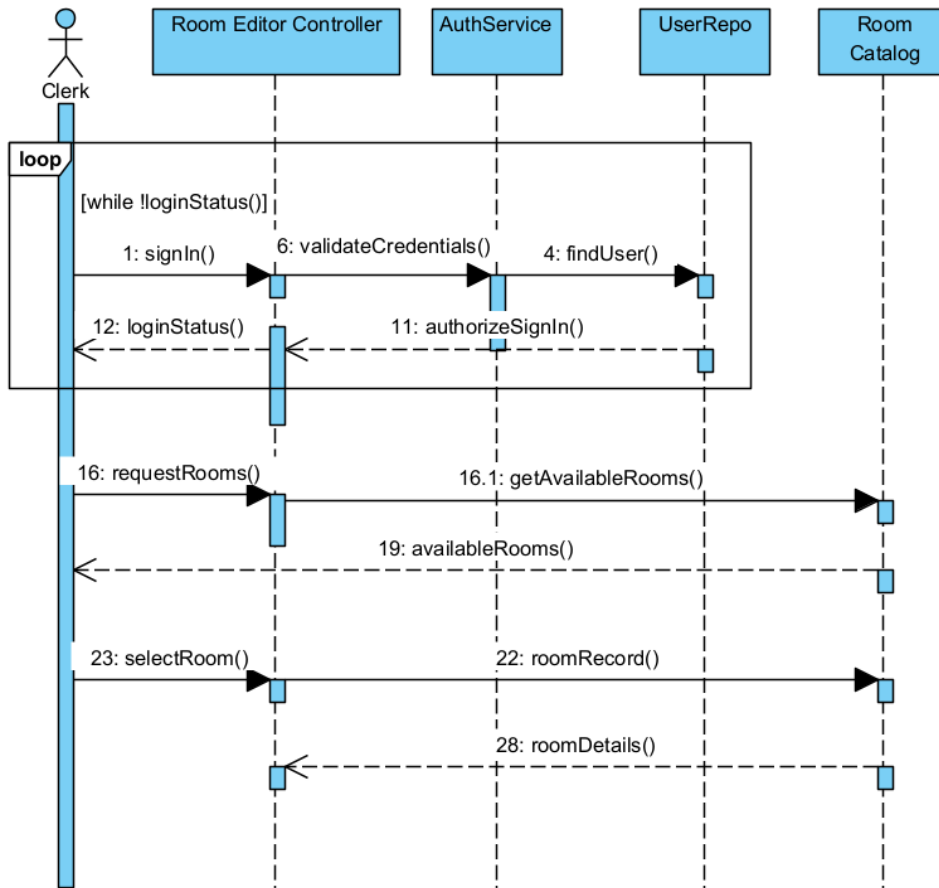
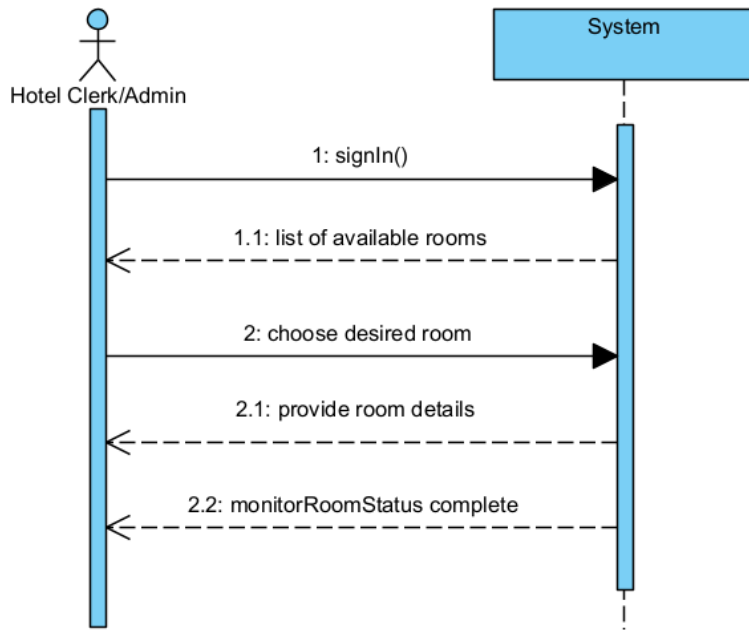
Use Case Name	Update Room Information
Use Case Author	Hannah Ross
Actor	Clerk
Preconditions	1. Clerk is logged in. 2. The system has access to all rooms, and corresponding details.
Postconditions	1. Details of room's information is updated. 2. Room is available to be reserved for future dates.
Main Success Scenario	1. Clerk selects room to update. 2. System provides all rooms and current details. 3. Clerk enters updated room information (ex. Quality level, smoking status) 4. System validates updated information. 5. System saves the changes and displays the confirmation message.
Extensions	1. Invalid search criteria (step 4) i. The system detects invalid input. ii. The system displays an error message and requests corrected input. iii. The room is currently occupied.

Contract CO1: updateRoomInfo
Operation: updateRoomInfo(room)
Cross References: Use Case – Sign in (clerk, admin)
Preconditions 4. The system is online. 5. Clerk/Admin authenticated. 6. Room to be selected is “unoccupied” and exists in system.
Postconditions 5. The room's attributes are updated. 6. The system updates the room with details.



Use Case Name	Monitor Room Status
Use Case Author	Hannah Ross
Actor	Clerk
Preconditions	1. Clerk is logged in. 2. The system has access to all rooms and corresponding details.
Postconditions	1. System provides a dashboard showing occupied, reserved, and vacant rooms of all 3 floors.
Main Success Scenario	1. Clerk requests the current status of all rooms. 2. System retrieves room information for all floors. 3. System displays each room's status (occupied, reserved, or vacant). 4. Clerk derives details necessary.
Extensions	1. System is down i. Provides incorrect information about rooms. 2. Room data is incomplete i. System detects information missing and notifies clerk.

Contract CO1: monitorRoomStatus
Operation: monitorRoomStatus(room)
Cross References: Use Case – Sign in (clerk, admin)
Preconditions 7. The system is online. 8. Clerk/Admin is authenticated. 9. Room to be selected is “unoccupied” and exists in system.
Postconditions 7. The room is found in the system. 8. The system displays the status of the room.

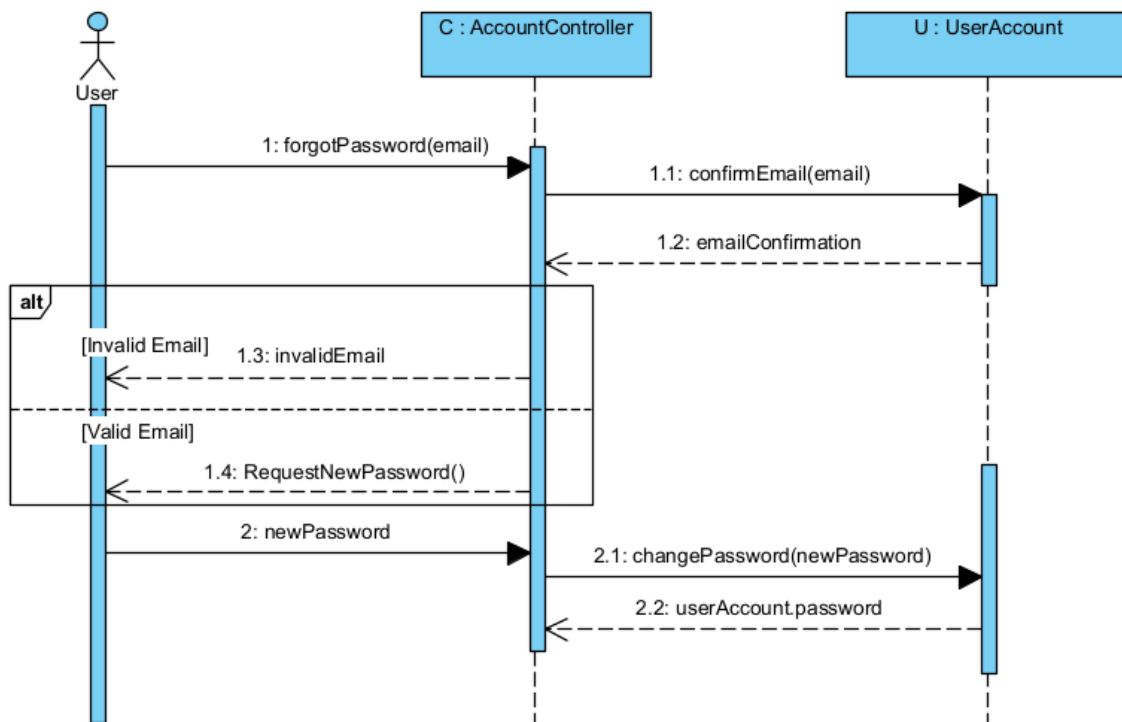
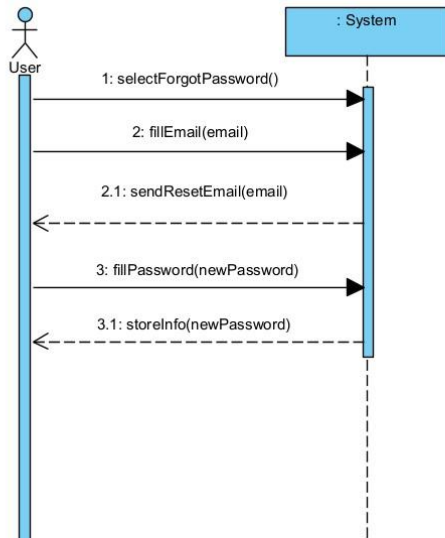


Emmanuel Humber's Use Cases

Use Case Name	Reset Password
Use Case Author	Emmanuel Humber
Actor	Guest / Admin
Preconditions	1. Guest has an account password
Postconditions	1. Guest account is assigned new password
Main Success Scenario	1. A guest clicks the forgot password button 2. The guest enters their account email 3. The guest gets an email with a link to reset their password 4. The guess enters new password and confirms it 5. The system updates the user's old password with a new one
Extensions	1. If Guest Forgets Account Email: i. If the guest does not remember their account email, they can click forgot email button. ii. The user is then prompted to call the hotel to receive further help from the admin. iii. The admin confirms the guest's identity through a series of questions. iv. Once admin has confirmed the guest's identity, he can select their account and replace their password with a password of the guest's choice and give them their account email.

Contract CO1: resetPassword Operation: resetPassword(newPassword) Cross References: Use Case – Reset Password Preconditions

Preconditions	1. System is online 2. User has a valid account
Postconditions	1. User password is replaced with typed newPassword 2. System stores information to user's account.



Use Case Name	Create Clerk Account
Use Case Author	Emmanuel Humber
Actor	Clerk
Preconditions	1. Clerk does not already have an account 2. Login system is up accessible
Postconditions	1. A new clerk account with the associated work email will be created. 2. Clerk will have an individualized clerk interface
Main Success Scenario	1. Clerk clicks create account button 2. Clerk enters work email and password prompted by system. 3. A new clerk account has been created. 4. System validates and stores clerk account information 5. Clerk is redirected to log in page.
Extensions	1. Account with email already detected - System detects accounts with a matching email. - Prompts user with email already in use error message. 2. System Error - If system is down, it will prompt system is currently down error message.

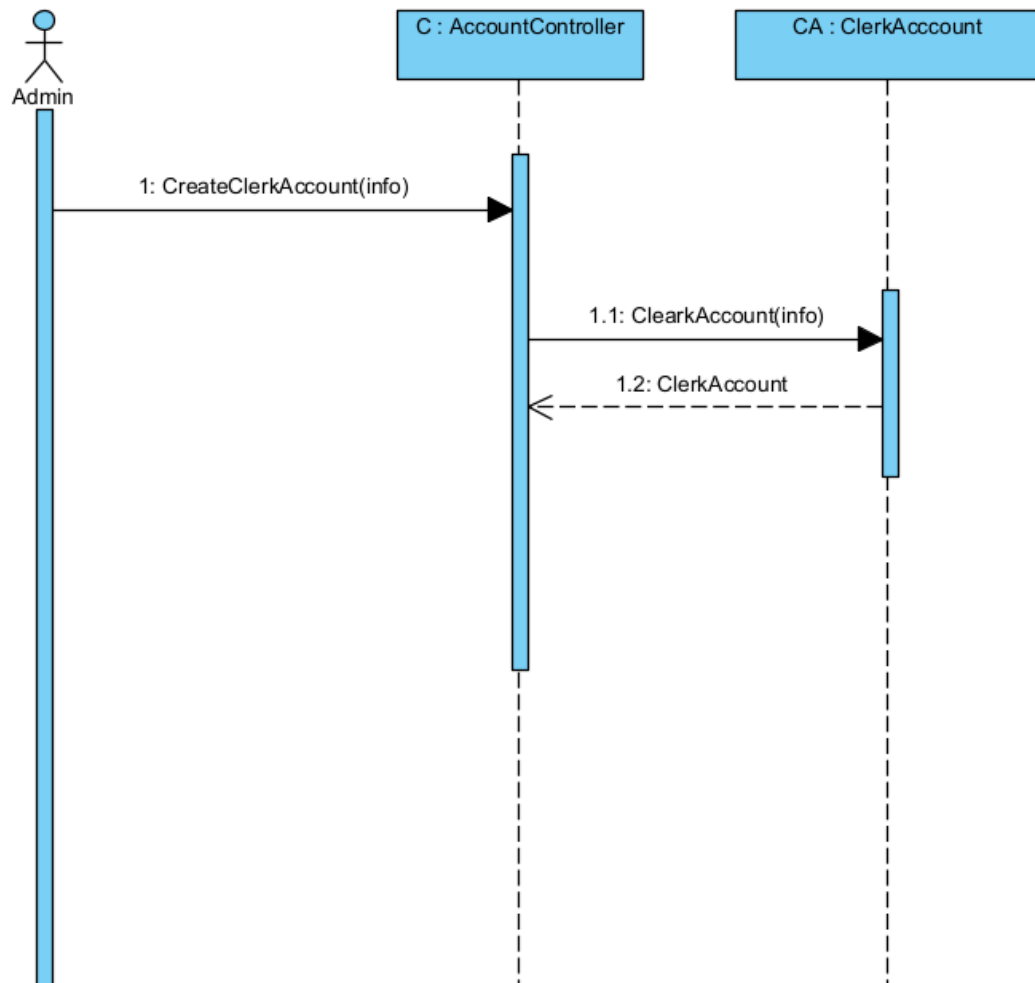
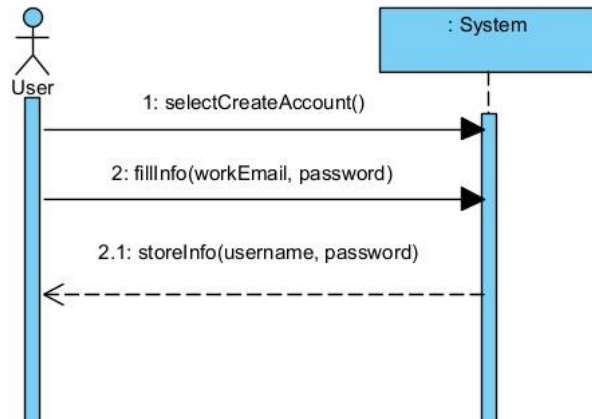
Contract CO1: createClerkAccount Operation: createClerkAccount(username, password) Cross

References: Use Case – Create Clerk Account Preconditions

- Clerk does not already have an account
- System is online

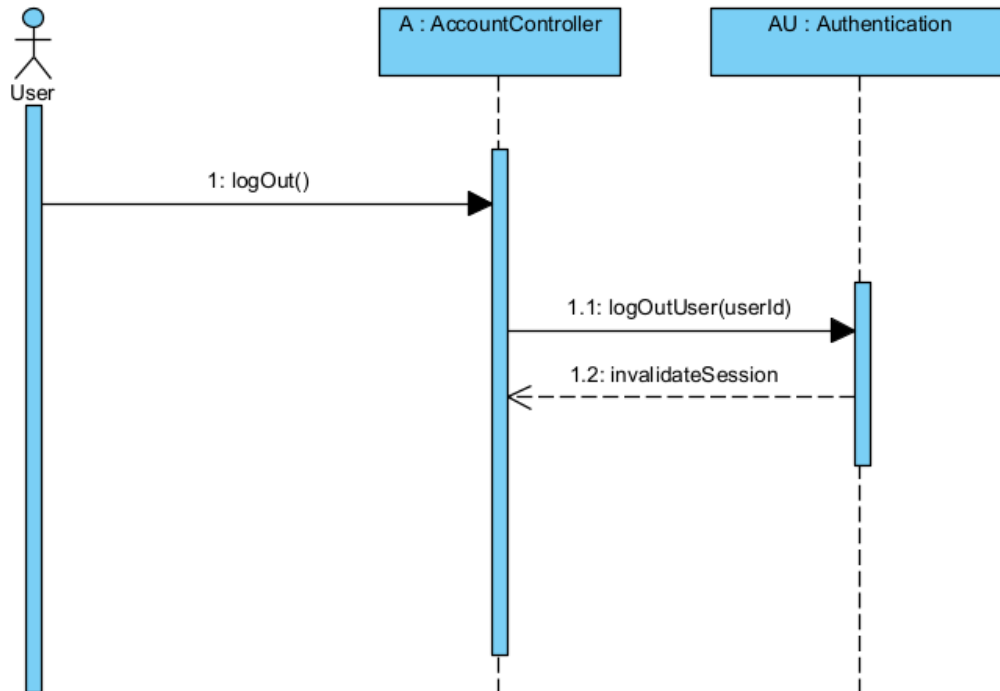
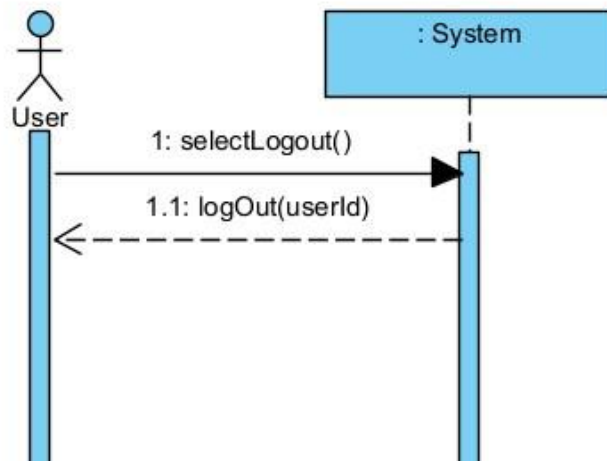
Postconditions

- Clerk is assigned account to his user with work email



Use Case Name	Log out
Use Case Author	Emmanuel Humber
Actor	Guest/Clerk
Preconditions	1. Guest is logged into valid hotel account. 2. System is up and running
Postconditions	1. Guest should be logged out of hotel account. 2. Guest should be redirected back to log in page.
Main Success Scenario	1. Guest navigates to settings. 2. Guest scrolls down to log out button. 3. Guest clicks log out and is redirected to log in page.
Extensions	1. System is down a. If system is not working an error message will pop and guest will not be logged out.

Contract CO1: logOut()
Operation: logOut()
Cross References: Use Case – Log In, Log Out
Preconditions 1. User is logged in their account 2. System is online
Postconditions 1. User is logged out of their account 2. Log in screen is displayed



Duane Schlottke's Use Cases

Use Case Name	Modify Reservation
Use Case Author	Duane Schlottke
Actor	Hotel Guest / Clerk
Preconditions	1. Valid reservation exists prior to start date. 2. User is logged in
Postconditions	1. User selects a reservation to modify. 2. System checks date of original reservation creation. 3. System checks date requested to be changed to 4. If date and time is available, fulfill request and cancel previous reservation 5. If date and time is not available, prompt user to either cancel or keep reservation
Main Success Scenario	1. User selects a reservation to cancel. 2. System checks date of original reservation creation. 3. System displays updated cost and update reservation details 4. User confirms. 5. System updates old status to Cancelled and new reservation to set
Extensions	1. User Aborts (step 5) i. Modification is canceled; reservation remains active.

Contract CO1: modifyReservation

Operation: modifyReservation(reservationId, newDate)

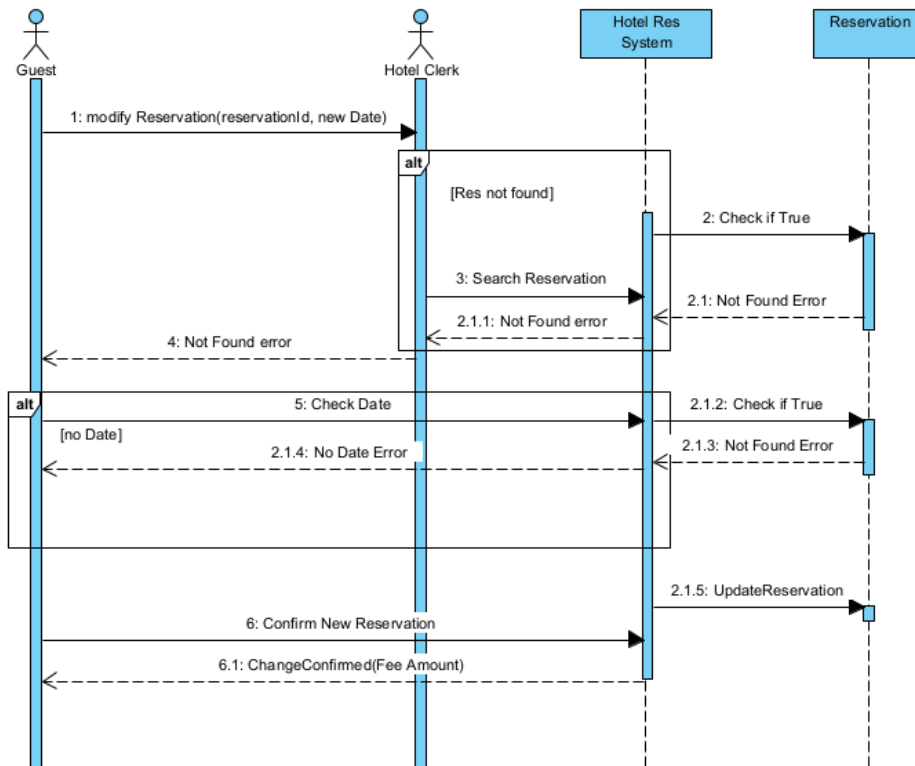
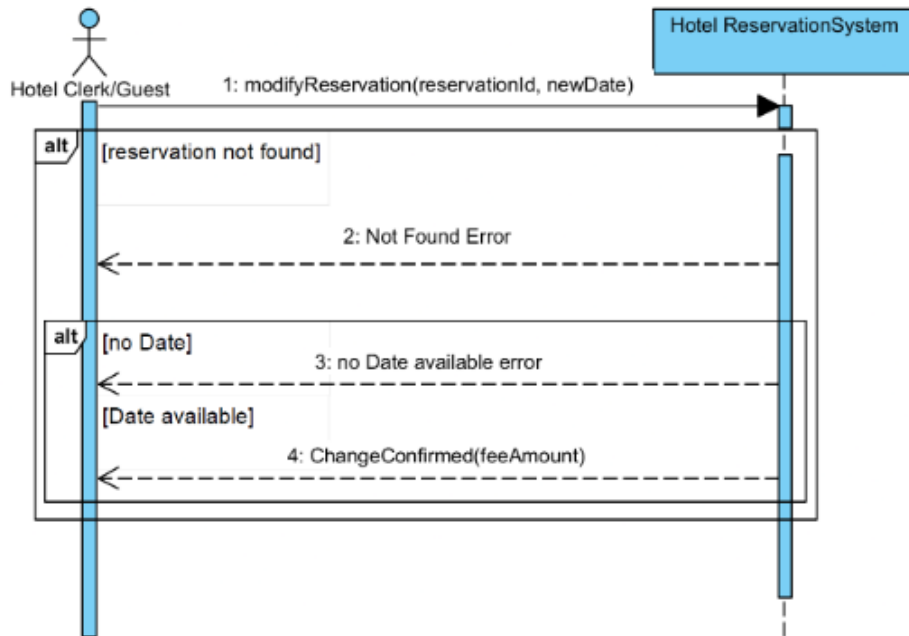
Cross References: Use Case – Sign in (clerk, guest)

Preconditions

1. The system is online and operational.
2. The actor is authenticated or otherwise uniquely identified by the system.
3. The requested stay details are valid (e.g., check-in date before check-out date).
4. Availability data exists (current reservations are accessible for availability checking).

Postconditions

1. A Reservation is created and stored.
2. It records the requested dates and is marked Booked.
3. It is associated with the Guest, selected RoomType, and a payment method.
4. A specific room is assigned (RoomType-level reservation).
5. The system returns a reservationId
6. The sytem marks the previous reservation as available



Use Case Name	Cancel Reservation
Use Case Author	Duane Schlotke
Actor	Hotel Guest / Clerk
Preconditions	1. Valid reservation exists prior to start date. 2. User is logged in 3. The cancellation occurs 2+ days to reservation date.
Postconditions	1. System deletes reservation on Guest's account 2. System updates room to available
Main Success Scenario	1. User selects a reservation to cancel. 2. System checks date of original reservation creation. 3. If > 2 days since creation, system calculates 80% single-night penalty. 4. System displays total refund/penalty and requests confirmation. 5. User confirms. 6. System updates status to Cancelled
Extensions	1. User Aborts (step 5) i. Cancellation is canceled; reservation remains active.

Contract CO1: cancelReservation

Operation: cancelReservation(reservationId, Date)

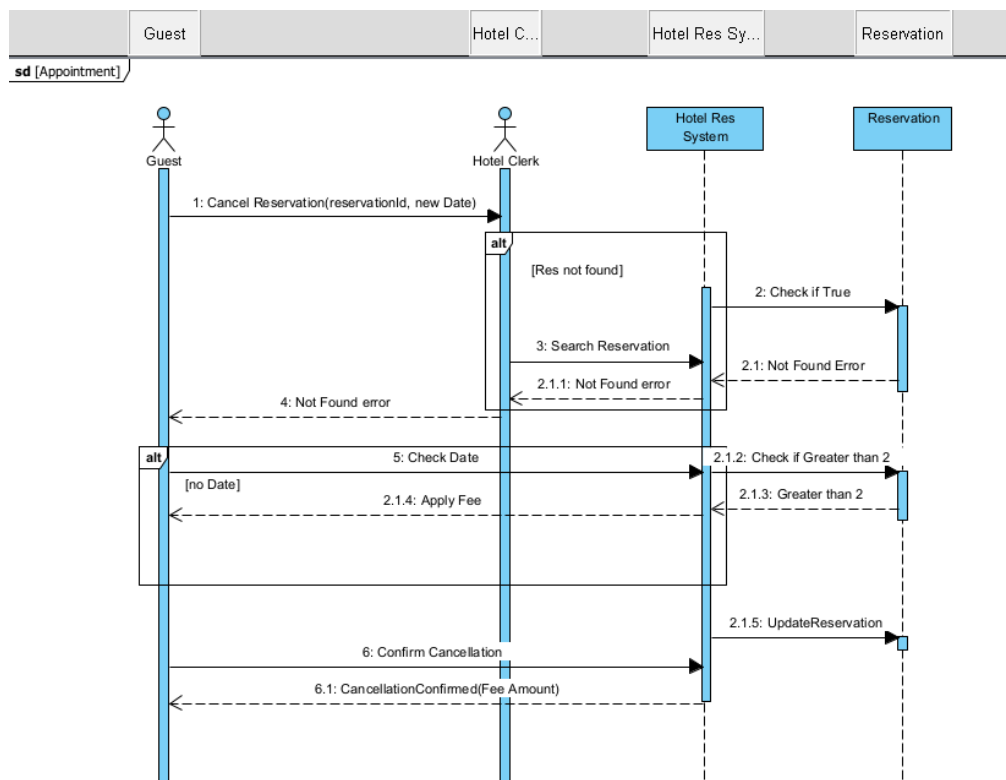
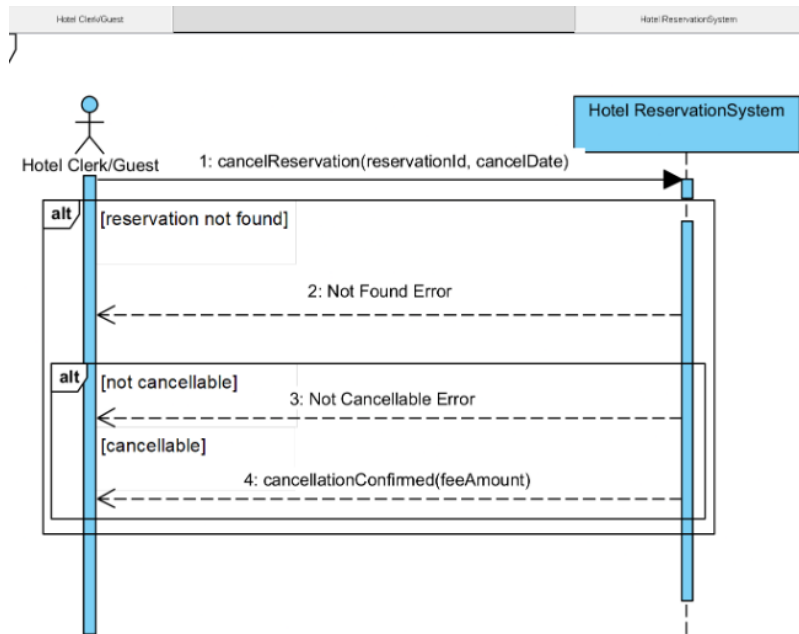
Cross References: Use Case – Sign in (clerk, guest)

Preconditions

5. The system is online and operational.
6. The actor is authenticated or otherwise uniquely identified by the system.
7. The requested stay details are valid (e.g., check-in date before check-out date).
8. Availability data exists (current reservations are accessible for availability checking).

Postconditions

7. A Reservation is created and stored.
8. It records the requested dates and is marked Booked.
9. It is associated with the Guest, selected RoomType, and a payment method.
10. No specific room is assigned yet (RoomType-level reservation).
11. The system returns a reservationId



Use Case Name	Add Guest to Reservation
Use Case Author	Duane Schlottke
Actor	Hotel Guest / Clerk
Preconditions	1. Valid reservation exists prior to start date. 2. User is logged in 3. Guest user has valid account
Postconditions	1. User selects a reservation to add guest to. 2. System verifies guest account exists 3. System displays new guest list 4. User confirms. 5. System updates guest list to reservation status
Main Success Scenario	1. User selects a reservation to add a guest to. 2. System verifies reservation creation and account validity. 3. System displays total new guest list 4. User confirms. 5. System updates guest list
Extensions	1. User Aborts (step 5) i. Guest add is cancelled, guest list remains unchanged

Contract CO1: addGuest

Operation: addGuest(room)

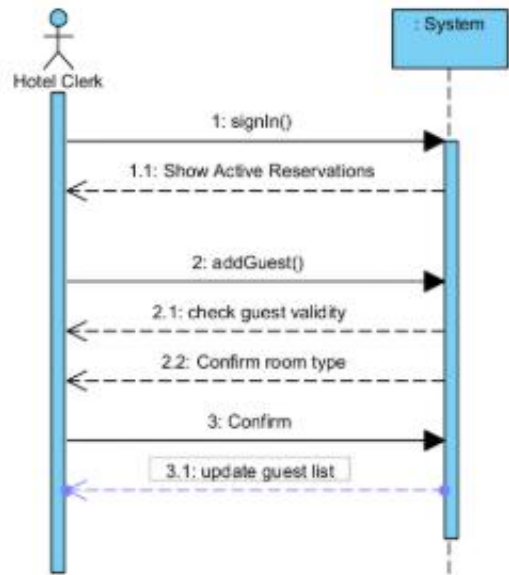
Cross References: Use Case – Sign in (clerk, guest)

Preconditions

1. The system is online.
2. Clerk/Guests authenticated.
3. Room to be selected is valid room type
4. Guest to be added is valid user

Postconditions

1. The guest is added to the reservation



sd [Appointment]

